

# Развёртывание системы BHS.CRG

Система исполнительной документации BHS.CRG

Версия документа: 2026-08-22

# Инструкция по развёртыванию BHS.CRG

Система генерации исполнительной документации для электромонтажных проектов. Развёртывание — через **Docker Compose** на сервере **Linux**: весь стек (веб-интерфейс, сервер приложений, база данных, объектное хранилище, модель распознавания) поднимается несколькими командами.

Этот документ — для администратора/инженера, разворачивающего систему. Для работы с системой см. «Инструкцию пользователя» и «Инструкцию администратора».

## 1. Архитектура развёртывания

| Компонент | Образ / технология          | Назначение                                                                                           |
|-----------|-----------------------------|------------------------------------------------------------------------------------------------------|
| web       | nginx + собранный React-SPA | Веб-интерфейс; проксирует /api на сервер                                                             |
| api       | ASP.NET Core 10 + Typst     | Сервер приложений, REST API, генерация PDF                                                           |
| postgres  | PostgreSQL 16               | Основная база данных                                                                                 |
| minio     | MinIO                       | Объектное хранилище (сканы, наборы данных, готовые файлы)                                            |
| ollama    | Ollama                      | Локальная модель распознавания сканов. По умолчанию не разворачивается — включается профилем, см. §7 |

Вспомогательные одноразовые сервисы: createbuckets (создаёт bucket в MinIO), ollama-init (загружает модель распознавания — вместе с ollama, только при включённом профиле).

Схематично:



**Генерация:** PDF собирается движком **Typst** (входит в образ api). Формат **DOCX в текущей версии не поддерживается** — только PDF.

## 2. Требования

- HTTPS обязателен.** Система работает с паролями и токенами доступа; по открытому HTTP они идут по сети в читаемом виде, и перехват одного запроса даёт полный доступ к системе от имени пользователя. Стек TLS не терминирует сам — перед сервисом web ставится обратный прокси с сертификатом (nginx, Caddy, Traefik) либо терминатор периметра. Без него допустима только установка, целиком закрытая внутри доверенной сети. См. §11.
- ОС:** Linux (Ubuntu 22.04+/Debian 12+ и совместимые).

- **Docker Engine 24+** и **Docker Compose v2 или новее** — тот, что вызывается как `docker compose` (через пробел), а не `docker-compose`. Номер у плагина давно ушёл вперёд движка: свежая установка отвечает `v5.x`, и это не ошибка — важно, что не `v1`. На чистом сервере их ещё нет: установка из официального репозитория — **Приложение А**. Там же сказано, чего не ставить (`docker.io`, `docker-compose v1`, `snar`) и почему выбор не того пакета выясняется не сразу.
- **Ресурсы:**
  - Базовый стек (без Ollama): от **2 CPU / 4 ГБ RAM / 10 ГБ диска**. Столько и требует установка по умолчанию — локальное распознавание не разворачивается, пока его не включат.
  - С Ollama и моделью `qwen2.5vl:7b` (включается по \$7): дополнительно **~8 ГБ RAM** и **~6 ГБ диска** под модель; желательна видеокарта, но работает и на CPU (медленнее).
- Доступ в интернет на этапе сборки (загрузка образов, Typst, npm/NuGet-пакетов) и, при использовании веб-поиска документов качества, в рантайме.

Проверка окружения:

```
docker --version
docker compose version
```

Обе команды должны ответить номером версии. Ответ `'compose' is not a docker command` означает, что плагин Compose не установлен: см. **Приложение А**.

### 3. Быстрый старт

Исходный код для установки **не нужен**: `api` и `web` поставляются готовыми образами из GHCR. Достаточно двух файлов со страницы выпуска.

```
# 1. Взять со страницы выпуска (https://github.com/pavru/bhs.crg/releases/latest)
#   docker-compose.yml и .env.example — они приложены к релизу.
mkdir bhs-crg && cd bhs-crg
curl -LO https://github.com/pavru/bhs.crg/releases/latest/download/docker-compose.yml
curl -Lo .env.example https://github.com/pavru/bhs.crg/releases/latest/download/env.example

# 2. Подготовить конфигурацию
cp .env.example .env
nano .env                # задать пароли, JWT-секрет и APP_VERSION (см. раздел 4)

# 3. Запустить весь стек (образы скачаются из GHCR)
docker compose up -d

# 4. Проверить статус
docker compose ps
docker compose logs -f api
```

Дальше в инструкции команды пишутся в виде `docker compose -f deploy/docker-compose.yml ...` — это форма для запуска из клона репозитория. Если вы поставили систему из релиза, файл лежит рядом, и `-f deploy/docker-compose.yml` можно опускать.

**Сборка из исходников** (разработка, проверка правки до релиза, установка без доступа к ghcr.io):

```
git clone https://github.com/pavru/bhs.crg.git && cd bhs.crg
cp deploy/.env.example deploy/.env && nano deploy/.env
docker compose -f deploy/docker-compose.yml -f deploy/docker-compose.build.yml up -d --build
```

Образы публичные — `docker login ghcr.io` для установки не нужен: пакеты наследуют видимость репозитория, и проверено это на первом же выпуске (анонимный pull обоих образов работает). Если однажды pull всё же потребует авторизации, значит видимость пакета изменили вручную — вернуть её можно в *Packages* → *Package settings* → *Change visibility*.

После запуска веб-интерфейс доступен по адресу **http://СЕРВЕР:8080** (порт задаётся `WEB_PORT` в `.env`).

При первом старте сервер `api` автоматически применяет миграции базы данных и создаёт роли `Admin` / `User`. Отдельных команд миграции выполнять не нужно.

---

## 4. Конфигурация ( `.env` )

---

Скопируйте `.env.example` → `.env` и заполните. Файл `.env` содержит секреты и **не** хранится в репозитории. При установке из релиза оба файла лежат в рабочем каталоге; в клоне репозитория — в `deploy/`.

| Переменная                                      | Назначение                                                                                                                                                                                                                                                                                                                                                                                                             | Обязательно                          |
|-------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------|
| APP_VERSION                                     | Версия системы: под этим тегом тянутся образы <code>api</code> и <code>web</code> из GHCR. Без неё стек не поднимется — умолчания намеренно нет, иначе установка молча работала бы на непонятной версии. Номер берётся со <a href="#">страницы выпусков</a> ; в шаблоне из релиза он уже проставлен                                                                                                                    | да                                   |
| POSTGRES_DB / POSTGRES_USER / POSTGRES_PASSWORD | Параметры БД. Пароль из примера останавливает запуск                                                                                                                                                                                                                                                                                                                                                                   | да (сменить пароль)                  |
| MINIO_ROOT_USER / MINIO_ROOT_PASSWORD           | Учётные данные MinIO — это <b>администратор хранилища</b> . Незаданные или взятые из примера останавливают запуск                                                                                                                                                                                                                                                                                                      | да (сменить <b>оба</b> )             |
| MINIO_BUCKET                                    | Имя bucket для файлов. Пустое значение останавливает запуск                                                                                                                                                                                                                                                                                                                                                            | да (по умолч. <code>bhs-crg</code> ) |
| MINIO_CONSOLE_PORT / MINIO_CONSOLE_BIND         | Порт и адрес публикации веб-консоли MinIO. По умолчанию <code>127.0.0.1</code> — только с самого сервера, см. §6                                                                                                                                                                                                                                                                                                       | нет                                  |
| JWT_KEY                                         | Секрет подписи токенов, <b>≥ 32 символов</b>                                                                                                                                                                                                                                                                                                                                                                           | да                                   |
| WEB_PORT                                        | Порт веб-интерфейса на хосте                                                                                                                                                                                                                                                                                                                                                                                           | нет (по умолч. <code>8080</code> )   |
| FORWARDED_TRUSTED_NETWORKS                      | Сети, чьему заголовку <code>X-Forwarded-For</code> сервер доверяет (CIDR через запятую). Пусто — петля и частные диапазоны. Негодная запись останавливает запуск                                                                                                                                                                                                                                                       | нет                                  |
| APP_PUBLIC_URL                                  | Публичный адрес системы (напр. <code>https://docs.example.ru</code> ) — подставляется ссылкой в письма                                                                                                                                                                                                                                                                                                                 | да, если нужна почта                 |
| BACKUP_MAX_ARCHIVE_MB                           | Предел размера резервной копии, МБ (16...10240). Ей же задаётся <code>client_max_body_size</code> веб-сервера — второе значение править не нужно. Негодное останавливает запуск                                                                                                                                                                                                                                        | нет (по умолч. <code>500</code> )    |
| COMPOSE_PROFILES                                | Какие необязательные сервисы поднимать; сегодня значение одно — <code>ollama</code> . Пусто — локальное распознавание не разворачивается                                                                                                                                                                                                                                                                               | нет (пусто)                          |
| OLLAMA_MODEL                                    | Модель локального распознавания. Пусто — на новой установке Ollama выключена и в самом приложении: движок участвует в работе ровно тогда, когда у него выбрана модель. Задаётся ВМЕСТЕ с <code>COMPOSE_PROFILES=ollama</code> . Оговорка для существующих установок: если настройки интеграций хоть раз сохраняли в интерфейсе, модель записана в базу, а база сильнее переменной — выключать движок тогда надо там же | нет (пусто)                          |
| ANTHROPIC_API_KEY                               | Распознавание реквизитов (Claude)                                                                                                                                                                                                                                                                                                                                                                                      | опц.                                 |

| Переменная     | Назначение                        | Обязательно |
|----------------|-----------------------------------|-------------|
| GEMINI_API_KEY | Распознавание реквизитов (Gemini) | опц.        |
| SERPER_API_KEY | Веб-поиск документов качества     | опц.        |

Сгенерировать надёжный JWT\_KEY :

```
openssl rand -base64 48
```

**Важно:** обязательно смените пароли БД/MinIO и JWT\_KEY перед вводом в эксплуатацию. Значения из примера — только для ознакомления.

JWT\_KEY проверяется при запуске: **приложение не поднимется**, если значение не задано, короче 32 байт или оставлено из файла-примера. Это сделано намеренно — иначе установку легко ввести в эксплуатацию, не заметив, что ключ подписи не менялся.

**Так же проверяются настройки хранилища и параметры БД.** Раньше при незадаанных значениях система поднималась молча, а отказ приходил позже и чужими словами: ошибка клиента MinIO при первой загрузке файла, Host can't be null от драйвера БД. Теперь запуск останавливается с указанием, что именно задать, если не заполнено любое из:

- MINIO\_ROOT\_USER , MINIO\_ROOT\_PASSWORD , MINIO\_BUCKET ;
- **любая часть** строки подключения — POSTGRES\_DB , POSTGRES\_USER , POSTGRES\_PASSWORD или адрес сервера. Строка разбирается, а не проверяется на пустоту целиком: незаданная переменная даёт не пустую строку, а строку с пустым полем ( ...;Password= ), и проверка «не пусто» такое пропустила бы — то есть ровно в том случае, с которым и сталкиваются, её бы не было.

Сюда же — BACKUP\_MAX\_ARCHIVE\_MB : не число или значение вне 16...10240 останавливает запуск. Понять «500 МБ» как 500 и промолчать было бы хуже, чем отказать: описка в .env тихо вернула бы умолчание на установке, где предел специально поднимали. Контейнер web на негодном значении тоже не поднимается.

Значение, оставленное из файла-примера, тоже останавливает запуск. Смотреть docker compose logs api .

Пароль БД обязателен: вход без пароля ( trust , .pgpass ) наша поставка не разворачивает, а СУБД, доступная по сети без пароля, — сама по себе дефект.

Если система уже работала на значении из примера, после смены ключа **отзовите refresh-токены**: выданные ранее перестанут быть доверенными, пользователи войдут заново. Это ожидаемое поведение.

## Почтовые сценарии (сброс пароля, подтверждение email)

Письма со **ссылками** — самостоятельный сброс пароля, подтверждение адреса, смена email, отправка крупного комплекта — требуют **двух** настроек:

1. APP\_PUBLIC\_URL в deploy/.env — внешний адрес, по которому пользователи открывают систему (именно он подставляется в ссылку письма). Без него письма со ссылками **не отправляются**, а действие возвращает понятную ошибку (пользователь бесполезного письма не получит).

2. **SMTP** — параметры почтового сервера задаёт администратор в UI: **Настройка системы** → **Настройки** → **Почта** (host/port/логин/шифрование). Без SMTP письма не уходят.

Ключи шифрования одноразовых токенов (сброс/подтверждение) хранятся на томе `dp_keys` (в `compose` уже настроен) — поэтому ссылки переживают перезапуск контейнера `api`. Срок жизни access-токена — 60 мин (обновляется автоматически); настраивается `Jwt__AccessTokenMinutes` при необходимости.

## 5. Первый вход (создание администратора)

1. Откройте **http://СЕРВЕР:8080**. Пока в системе нет ни одного пользователя, вместо формы входа открывается экран **«Первый вход»**: email, имя, пароль и его подтверждение.
2. Заполните и нажмите **«Создать администратора и войти»**. Учётная запись получит роль **Администратор**, и вход выполнится сразу — второй раз вводить те же данные не нужно.
3. После этого **публичная регистрация закрывается сама**: по тому же адресу снова обычная форма входа. Остальные учётные записи заводит администратор — **Настройка системы** → **Пользователи** (см. «Инструкцию администратора»).

**Это окно открыто всем.** От `ip -d` и до создания первой учётной записи экран «Первый вход» доступен любому, кто дотянется до адреса, и администратором станет тот, кто успел первым. Делайте первый вход сразу после запуска — а если система смотрит наружу, то **до** того, как открыть к ней доступ.

**Если к интерфейсу доступа нет, а к хосту есть** (ставите по SSH, порт наружу ещё закрыт), администратора можно завести запросом — это тот же путь, которым идёт форма:

```
curl -i -X POST http://localhost:8080/api/auth/register -H "Content-Type: application/json"
-d '{"email":"admin@example.ru","password":"ВашПароль1","displayName":"Администратор"}'
```

Пароль — не короче 8 символов, с цифрой, строчной и заглавной буквой. `-i` здесь обязателен: при успехе тело ответа **пустое**, и без кода состояния удача неотличима от молчаливого отказа. `200 OK` — готово; `400` — в теле сказано, чем плох пароль; `403` — учётная запись уже есть.

## 6. Порты

| Сервис          | Внутренний порт | Порт на хосте                             | Примечание                                   |
|-----------------|-----------------|-------------------------------------------|----------------------------------------------|
| web (nginx)     | 8080            | <code>WEB_PORT</code> (8080)              | Основной вход в систему                      |
| api             | 8080            | —                                         | Доступен только внутри сети Compose          |
| postgres        | 5432            | —                                         | Наружу не публикуется                        |
| minio (API)     | 9000            | —                                         | Наружу не публикуется                        |
| minio (консоль) | 9001            | <code>127.0.0.1:MINIO_CONSOLE_PORT</code> | Админ-консоль MinIO, только с самого сервера |
| ollama          | 11434           | —                                         | Наружу не публикуется                        |

Наружу выведен только веб-интерфейс. Базу, хранилище и Ollama публиковать не следует.

**Консоль MinIO** публикуется на петлю ( `MINIO_CONSOLE_BIND` , по умолчанию `127.0.0.1` ): она открывает управление хранилищем под учётной записью `MINIO_ROOT_USER` , то есть второй вход ко всем файлам системы в обход приложения и его ролей. С рабочей машины на неё ходят через SSH-туннель:

```
ssh -L 9001:127.0.0.1:9001 пользователь@сервер # затем http://localhost:9001
```

Внутренний порт `web` — 8080, а не 80: nginx в образе работает от непривилегированного пользователя. Порт на хосте ( `WEB_PORT` ) прежний, для обратного прокси ничего не меняется.

## Заголовки безопасности

Сервис `web` отдаёт `Content-Security-Policy` , `X-Content-Type-Options` , `X-Frame-Options` , `Referrer-Policy` , `Permissions-Policy` и `Strict-Transport-Security` (см. `deploy/nginx.conf` ). Политика содержимого разрешает скрипты **только с адреса самого приложения** — сторонние CDN и внедрённые в данные инлайновые скрипты не выполняются.

Если вы правите фронтенд под себя и подключаете стороннюю библиотеку по ссылке, она будет заблокирована — это ожидаемо. Правильный путь: положить библиотеку в сборку, а не ослаблять политику. HSTS браузер применяет только поверх HTTPS, поэтому заголовок безвреден до появления терминатора TLS и включается сам, когда тот появится.

## 7. Опциональные компоненты

### Распознавание сканов (документы качества)

- **Ollama (локально):** по умолчанию **не разворачивается** (issue #824) — образ с моделью занимает ~6 ГБ диска и до 10 ГБ памяти, а установке с облачным распознаванием (или вовсе без документов качества) они не нужны. Включается двумя строками в `.env` и обычным `up -d` :



```
# в .env раскомментировать ОБЕ:
COMPOSE_PROFILES=ollama      # поднимать контейнеры ollama и ollama-init
OLLAMA_MODEL=qwen2.5vl:7b     # какую модель качать и какой пользоваться приложению

docker compose -f deploy/docker-compose.yml up -d
docker compose -f deploy/docker-compose.yml logs -f ollama-init # загрузка модели, ~6 ГБ
```

Если ваша сборка Compose не подхватывает `COMPOSE_PROFILES` из `.env` (ранние выпуски v2 читали эту переменную только из окружения), то же самое даёт флаг: `docker compose -f deploy/docker-compose.yml --profile ollama up -d`. Признаком именно этой беды один: `up -d` отработал без единой ошибки, а контейнеров `ollama` в `ps` нет.

Что включилось, видно в интерфейсе: **Настройка системы** → **Настройки** → **Поиск и распознавание** (Ollama перестаёт числиться ненастроенной) и в состоянии системы — появляется компонент «Ollama (распознавание)».

Загрузить или сменить модель вручную:

```
docker compose -f deploy/docker-compose.yml exec ollama ollama pull qwen2.5vl:7b
```

**Выключить обратно:** закомментировать обе строки в `.env` — и убрать контейнеры **явно**:

```
docker compose -f deploy/docker-compose.yml rm -sf ollama ollama-init
```

И если настройки интеграций когда-либо сохраняли в интерфейсе — снимите Ollama там же (**Настройка системы** → **Настройки** → **Поиск и распознавание**): сохранённая модель лежит в базе, а база сильнее переменной окружения. Иначе приложение продолжит считать движок включённым и мониторинг будет исправно сообщать, что он не отвечает.

Одной правки `.env` мало, и это проверено: контейнер отключённого профиля не останавливает ни `docker compose up -d` (даже с `--remove-orphans`), ни `docker compose down`. Последний уберёт остальные контейнеры, споткнётся о сеть (`Resource is still in use`) и оставит `ollama` работать; на стенде `down -v` снёс всё прочее вместе с томами, а она продолжила работать как ни в чём не бывало. Профиль решает, что поднимать, а не что гасить — молча остановленный им контейнер выглядел бы выключенным, а память и диск занимал бы дальше. Пока контейнер существует, он живёт своей жизнью: с политикой `restart: unless-stopped` он вернётся и после перезагрузки хоста. Том `ollama_data` со скачанной моделью тоже остаётся — освободить место отдельно: `docker volume rm bhs-crg_ollama_data` (имена — `docker volume ls`).

Строки именно две, и они про разное: первая решает, поднимать ли контейнеры, вторая — берёт ли Ollama в работу приложение. Раскомментировать одну — не поломка, но лишний круг: только профиль даёт контейнер вхолостую (скажет `docker compose logs ollama-init`), только модель — предупреждение в колокольчике «Ollama (распознавание): недоступна», потому что приложение ждёт движок, которого на хосте нет.

- **Облачные модели:** задайте `ANTHROPIC_API_KEY` и/или `GEMINI_API_KEY` в `.env`. Порядок перебора движков настраивается в коде ( `Recognition:Order` ) и в разделе настроек интеграций.

## Веб-поиск документов качества

- Задайте `SERPER_API_KEY` (`serper.dev`) в `.env`. Без ключа поиск в интернете недоступен, остальная работа с документами качества — да.

Ключи интеграций можно также задавать/менять в самом приложении: **Настройка системы** → **Настройки** → **Поиск и распознавание**. В базе они хранятся **зашифрованными** (ключами Data Protection с тома `dp_keys` ) и маскируются при чтении через API. Ключи, заданные переменными окружения, шифровать нечем и не нужно — они и так вне базы.

Следствие: при потере тома `dp_keys` расшифровать сохранённые ключи и пароль SMTP не получится — их надо будет ввести заново. Пока том просто недоступен (не примонтирован, переехал путь), сохранённые значения не портятся: приложение покажет их как незаданные, но перезаписывать не станет — вернёт том, и они снова читаются.

**Пароль SMTP привязан к серверу.** Смена хоста, порта, пользователя или снятие шифрования обнуляют сохранённый пароль: его надо ввести заново. Это не придирка — иначе сохранение чужого адреса с пустым полем пароля отправило бы туда сохранённый пароль с первым же письмом. По той же причине проверка связи с другим сервером требует ввести пароль явно.

## 8. Обновление системы

Обновление выполняется **на хосте, из командной строки**. Кнопки «Обновить» в интерфейсе нет и не будет — это решение, а не недоделка: чтобы приложение пересобирало собственный стек, контейнеру нужен доступ к сокету Docker, а это превращает взлом веб-части во взлом всего сервера. Разбор с проверками — [issue #814](#).

Порядок ниже устроен так, что всё, что может отказать — загрузка образов, разбор нового `compose`-файла, недостающая переменная, — происходит **до** первой остановки. Дойдя до `up -d`, вы уже знаете, что обновление состоится.

**Где выполнять.** Команды написаны для установки из релиза: `docker-compose.yml` и `.env` лежат в текущем каталоге. Если система поставлена из клона репозитория, эти файлы — в `deploy/`: перейдите туда ( `cd deploy` ), и всё остальное совпадёт дословно.

**Кого прервёт.** Перезапуск обрывает фоновые задачи: незавершённая сборка комплекта или распознавание сканов сами не возобновятся, новая версия при старте пометит их неудавшимися. Пилуля задач в интерфейсе для проверки **не годится** — она показывает только ваши собственные задачи, а мешает как раз чужая. Спросить надо базу:

```
docker compose exec -T postgres psql -U postgres -d bhs_crg \  
-c "select \"Status\", count(*) from jobs where \"Status\" in ('Queued', 'Running') group by  
1;"
```

Пустой ответ — можно обновляться. ( `postgres` и `bhs_crg` — значения `POSTGRES_USER` и `POSTGRES_DB` из вашего `.env`.)

**Сколько займёт.** На стенде обновление 0.137.0 → 0.140.0 заняло 19 секунд, из них система была недоступна около 15: сперва API отвечал `502` (веб-часть ещё старая, `api` уже пересоздаётся), потом на несколько секунд переставал открываться и сам адрес, пока пересоздавался `web`. Тяжёлая миграция на большой базе это время удлинит — насколько, заранее не знает никто.

```
# 1. РЕЗЕРВНАЯ КОПИЯ — до всего остального (раздел 9). Схема базы мигрирует при старте новой
# версии, и отката миграций нет: вернуть прошлый образ можно, прошлую схему — только из
# копии.

# 2. Выбрать версию: https://github.com/pavru/bhs.crg/releases
VER=0.140.0

# 3. Забрать файлы ИМЕННО ЭТОЙ версии, во временный каталог — не поверх своих. Compose-файл
# приглашает к правкам (лимит сpus, порты, свои тома), и `curl -LO` рядом с ним унёс бы их
# молча.
mkdir -p new && cd new
curl -LO https://github.com/pavru/bhs.crg/releases/download/v$VER/docker-compose.yml
curl -Lo env.example https://github.com/pavru/bhs.crg/releases/download/v$VER/env.example
cd ..

# 4. Сохранить то, что работает сейчас: без этих копий откатываться будет не на что.
cp docker-compose.yml docker-compose.yml.prev
cp .env .env.prev

# 5. Сравнить и перенести. Новые версии добавляют переменные, тома и лимиты; обновив только
# образы, вы получите новую настройку в её умолчании и не узнаете об этом.
diff docker-compose.yml new/docker-compose.yml # слева ваши правки, справа изменения версии
diff .env.example new/env.example              # что добавилось в шаблон окружения
cp new/docker-compose.yml docker-compose.yml    # свои правки, если они были, вернуть руками
cp new/env.example .env.example

# 6. Указать версию и добавить то, что появилось в шаблоне
nano .env                                     # APP_VERSION=$VER

# 7. Предполётная проверка — ничего ещё не останавливая
docker compose config -q                     # молчание = файл разобран и APP_VERSION задан
docker compose pull                          # образы новой версии уже на хосте

# 8. Обновиться (в соседней консоли полезно держать `docker compose logs -f api`)
docker compose up -d

# 9. Проверить
docker compose ps                           # api и web — running, api — healthy
curl -s http://localhost:8080/api/version
```

Шаг 5 — про **ваши** правки, а не про новые строки версии. Отказ здесь тихий: заменив файл целиком, вы не получите ошибки. Если менялось, например, имя проекта ( `name`: в первой строке), `docker compose ps` после замены вернёт пустой список — работающая система просто перестанет быть видна командам, а `up -d` поднимет рядом вторую, пустую установку. Проверено на стенде ровно так и вышло: ноль контейнеров в списке при живой системе.

Шаг 7 — не формальность, но и не полная проверка. `config -q` ручается за то, что `compose`-файл разобран и `APP_VERSION` задан (только эта переменная объявлена обязательной); пустые `JWT_KEY` или `POSTGRES_PASSWORD` он пропустит молча, они выяснятся уже при старте. `pull` же до `up -d` означает, что сорвавшаяся загрузка образа — недоступный `ghcr.io`, оборванная сеть — оставит систему работать на прежней версии. Оба отказа стоят ноль простая, если случились здесь.

Анонимный ответ `/api/version` показывает только номер версии: хеш коммита и дату сборки система отдаёт вошедшему пользователю — они видны внизу боковой панели интерфейса ( `v0.140.0 · a1b2c3d`, дата сборки в подсказке). Пустое поле `commit` в ответе `curl` — не признак неправильно собранного образа.

Миграции БД применяются автоматически при старте `api`. Данные сохраняются в томах (`postgres_data`, `minio_data`, `ollama_data`) и обновление их не трогает.

### Если новая версия не поднялась.

```
docker compose ps -a          # кто не в running
docker compose logs --tail=100 api  # причина почти всегда в первых строках
```

`up -d` в этом случае не завершится быстро: `web` ждёт готовности `api` по `healthcheck`, а тот сдаётся не сразу — 20 секунд на старт плюс 30 попыток по 10 секунд, то есть до пяти с лишним минут молчания, прежде чем появится строка вида `dependency failed to start: container bhs-crg-api-1 is unhealthy`. Прерывать не нужно; ради этого и держат вторую консоль с `logs -f api` — там причина видна сразу. Пока `api` не отвечает, старая веб-часть продолжает работать и отдаёт `502` на любой запрос к API — сказать о причине ей нечем, причина в журнале `api`. Самые частые: не хватило переменной окружения, которую добавила новая версия (пропущен шаг 5), и оборвавшаяся миграция.

**Откат.** Пока миграции не применились (в журнале `api` нет строк `Applying migration`), достаточно вернуть сохранённое на шаге 4 и повторить обновление в обратную сторону:

```
cp docker-compose.yml.prev docker-compose.yml
cp .env.prev .env
docker compose up -d
```

Если миграции применились — старый образ с новой схемой работать не обязан, и откат делается только восстановлением резервной копии (раздел 9), причём **базу перед этим пересоздают**: дампы, залитый поверх мигрировавшей схемы, оставит смесь новой схемы со старыми данными. Другого пути нет — поэтому копия из шага 1 и не обсуждается.

**Разово при обновлении с версии ниже 0.139.0 — про локальную Ollama.** Она больше не поднимается по умолчанию (профиль Compose, issue #824). На уже работающей установке обновление при этом ничего не выключает: `up -d` контейнер отключённого профиля не трогает, Ollama продолжит работать как работала. Выбор делается один раз:

- **пользуетесь ею** — раскомментируйте в `.env` `COMPOSE_PROFILES=ollama` и `OLLAMA_MODEL=...` (§7). Сама по себе она никуда не денется — переживает и `up -d`, и `down`, и перезагрузку хоста, — но исчезнет навсегда в день, когда контейнер уберут: явным `rm`, чисткой `docker system prune`, переносом на другой сервер. Профиль выключен, и обратно она не поднимется — а обнаружится это не сразу и не там;
- **не пользуетесь** — уберите её и освободите память и диск: `docker compose -f deploy/docker-compose.yml rm -sf ollama ollama-init` — именно `rm -sf`, потому что обычный `down` контейнер отключённого профиля не забирает (проверено). И проверьте **Настройка системы** → **Настройки** → **Поиск и распознавание**: на установке, где настройки хоть раз сохраняли, выбранная модель хранится в базе и очистку `.env` переживает — движок надо выключить там, иначе мониторинг будет вечно сообщать, что Ollama не отвечает.

Том `ollama_data` в обоих случаях цел: модель заново не качается. Проверено обновлением стенда 0.137.0 → 0.140.0: контейнер `ollama` после `up -d` остался работать как работал.

**Разово при переходе на поставку образами (с версии ниже 0.137.0).** Прежде `api` и `web` собирались на хосте из исходников (`up -d --build`), теперь тянутся из GHCR. Тома, имена сервисов и данные те же; достаточно задать `APP_VERSION` в `.env` и выполнить `docker compose pull && docker compose up -d`. Локально собранные образы после этого можно удалить (`docker image prune`). Обратный путь — сборка из исходников — остаётся: `-f deploy/docker-compose.yml -f deploy/docker-compose.build.yml up -d --build`.

**Разово при обновлении с версии ниже 0.91.0.** Контейнеры перестали работать от `root`. Том `dp_keys` (ключи шифрования одноразовых ссылок) создавался от `root`, и владелец у существующего тома сам не поменяется — сервер сможет читать ключи, но не сможет выписать очередной, и через несколько недель ссылки сброса пароля начнут отказывать. Выполните один раз (`--entrypoint chown` обязателен: без него аргументы достанутся серверу приложений, он запустится от `root` — и ничего не поменяет, притом без единой ошибки):

```
docker compose -f deploy/docker-compose.yml run --rm --user root --entrypoint chown api -R app:app /app/dp-keys
```

На новой установке делать ничего не нужно: пустой том наследует владельца из образа.

## 9. Резервное копирование и восстановление

### База данных:

```
# Бэкап
docker compose -f deploy/docker-compose.yml exec -T postgres \
  pg_dump -U postgres --clean --if-exists bhs_crg > backup_$(date +%F).sql

# Восстановление. База ПЕРЕСОЗДАЁТСЯ, а не заливается поверх, и это проверено: дамп, наложенный
# на схему, которую уже изменили миграции новой версии, восстанавливает свои объекты и молча
# оставляет чужие – таблицы, добавленные новой версией, остаются, а список применённых миграций
# возвращается к старому. Ошибок при этом ноль, выглядит как успех, а следующий старт api
# упирается
# в попытку применить миграцию поверх уже существующих таблиц.
docker compose -f deploy/docker-compose.yml stop api
docker compose -f deploy/docker-compose.yml exec -T postgres psql -U postgres -d postgres \
  -c 'DROP DATABASE bhs_crg WITH (FORCE);' -c 'CREATE DATABASE bhs_crg OWNER postgres;'
cat backup_YYYY-MM-DD.sql | docker compose -f deploy/docker-compose.yml exec -T postgres \
  psql -U postgres -d bhs_crg
docker compose -f deploy/docker-compose.yml start api
```

Порядок важен: api останавливают **до** пересоздания базы и запускают **после**. Миграции он применяет при старте — подключившись к восстановленной базе, версия новее той, из которой снята копия, тут же мигрирует её обратно вперёд, и восстановление окажется отменено. Поэтому, восстанавливаясь после неудачного обновления, сперва верните в .env прежний APP\_VERSION.

**Файлы (MinIO):** резервируйте том `minio_data` (или зеркалируйте bucket через `mc mirror`).

В приложении также есть встроенное **резервное копирование** для администратора (**Настройка системы** → **Настройки** → **Резервное копирование**).

## 10. Диагностика

| Симптом                                                                                                                      | Причина / решение                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
|------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>docker compose</code> отвечает <code>'compose' is not a docker command</code>                                          | Docker поставлен без плагина Compose v2 — обычно это пакет <code>docker.io</code> из репозитория дистрибутива. Переставьте из официального репозитория: Приложение А                                                                                                                                                                                                                                                                                                                                  |
| <code>api</code> падает при старте                                                                                           | Обязательное значение не задано или оставлено из примера — сообщение в логе называет, какое именно и какой переменной задаётся: <code>JWT_KEY</code> , <code>MINIO_ROOT_USER</code> , <code>MINIO_ROOT_PASSWORD</code> , <code>MINIO_BUCKET</code> , параметры БД. Проверьте также доступность <code>postgres</code> (healthcheck).<br><code>docker compose logs api</code>                                                                                                                           |
| Генерация PDF с ошибкой шрифтов                                                                                              | В образ включены DejaVu/Liberation/Noto. Если шаблон требует особый шрифт — добавьте его в <code>Dockerfile.api</code>                                                                                                                                                                                                                                                                                                                                                                                |
| Распознавание не работает                                                                                                    | Локальная Ollama по умолчанию не разворачивается — включается профилем (§7). Иначе: модель не загружена ( <code>ollama pull</code> ) или не задан API-ключ облачного движка                                                                                                                                                                                                                                                                                                                           |
| В состоянии системы «Ollama (распознавание): недоступна», хотя локальное распознавание не включали                           | Приложение считает движок включённым, а контейнера на хосте нет. Либо в <code>.env</code> задан <code>OLLAMA_MODEL</code> без <code>COMPOSE_PROFILES=ollama</code> , либо модель сохранена в базе из интерфейса (она сильнее переменной). Задайте профиль — или выключите движок в <b>Настройки → Поиск и распознавание</b>                                                                                                                                                                           |
| Раскомментировал <code>COMPOSE_PROFILES=ollama</code> , <code>up -d</code> без ошибок, а контейнеров <code>ollama</code> нет | Сборка Compose не читает эту переменную из <code>.env</code> (ранние выпуски v2 — только из окружения). Тот же результат даёт флаг: <code>docker compose --profile ollama up -d</code>                                                                                                                                                                                                                                                                                                                |
| Профиль <code>ollama</code> включён, контейнер работает, а распознавание его не берёт                                        | Не задан <code>OLLAMA_MODEL</code> : движок без выбранной модели в перебор не попадает. В настройках он помечен «не выбрана модель», то же скажет <code>docker compose logs ollama-init</code>                                                                                                                                                                                                                                                                                                        |
| Убрал <code>COMPOSE_PROFILES=ollama</code> , а контейнер <code>ollama</code> всё работает                                    | Так и задумано Compose: профиль решает, что поднимать, а не что останавливать — <code>up -d</code> (и <code>--remove-orphans</code> ) запущенный контейнер не трогает. Уберите явно: <code>docker compose rm -sf ollama ollama-init</code>                                                                                                                                                                                                                                                            |
| Веб-интерфейс открывается, но «не видит» сервер                                                                              | Проверьте, что сервис <code>web</code> проксирует на <code>api:8080</code> (см. <code>deploy/nginx.conf</code> )                                                                                                                                                                                                                                                                                                                                                                                      |
| Вход отвечает «слишком много попыток входа»                                                                                  | Сработал предел частоты по адресу клиента. Если за одним внешним адресом работает целый офис — так и задумано. Если предел ловят единичные пользователи, значит адрес клиента до сервера не доходит: проверьте, что каждый прокси в цепочке дописывает <code>X-Forwarded-For</code> (в поставляемом <code>nginx.conf</code> это <code>\$proxy_add_x_forwarded_for</code> ), а сети всех прокси попадают в <code>FORWARDED_TRUSTED_NETWORKS</code> — иначе адресом клиента станет прокси, один на всех |
| Ссылки сброса пароля перестали работать после обновления                                                                     | Права на том <code>dp_keys</code> — см. разовую команду в §8                                                                                                                                                                                                                                                                                                                                                                                                                                          |



| Симптом                                                                                                         | Причина / решение                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
|-----------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>api</code> или <code>ollama</code> перезапускаются без внятной причины, в логах обрыв на тяжёлой операции | Упёрлись в потолок памяти контейнера. Поднимите <code>API_MEMORY_LIMIT</code> / <code>OLLAMA_MEMORY_LIMIT</code> в <code>.env</code> . Проверить: <code>docker stats</code> и <code>docker inspect --format '{{.State.OOMKilled}}'</code> <контейнер>                                                                                                                                                                                                                                                                                                                                                                      |
| Вёрстка PDF отвечает «не завершилась за 60 с»                                                                   | Шаблон зациклился либо рекурсия без выхода. Процесс останавливается намеренно: без срока он занимал бы ядро до перезапуска сервера                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| Не загружаются крупные файлы                                                                                    | Наибольший предел — архив резервной копии, и задаётся он одной переменной <code>BACKUP_MAX_ARCHIVE_MB</code> : из неё же считается <code>client_max_body_size</code> при старте <code>web</code> (см. <code>deploy/nginx-body-size.sh</code> ). Два числа в разных файлах здесь держать нельзя — порог <code>nginx</code> ниже нашего делает предел приложения недостижимым, а отказ приходит HTML-страницей <code>nginx</code> без поля <code>error</code> , и интерфейс показывает голое сообщение <code>axios</code> . Проверить действующее значение: <code>docker compose logs web   grep client_max_body_size</code> |
| Восстановление отвечает «Файл превышает N МБ»                                                                   | Копия переросла предел. Текущий вес против предела виден в интерфейсе: <b>Настройки</b> → <b>Резервное копирование</b> . Поднимите <code>BACKUP_MAX_ARCHIVE_MB</code> и пересоздайте <code>api</code> и <code>web</code>                                                                                                                                                                                                                                                                                                                                                                                                   |

Полезные команды:

```
docker compose -f deploy/docker-compose.yml ps           # статус
docker compose -f deploy/docker-compose.yml logs -f api  # логи сервера
docker compose -f deploy/docker-compose.yml down         # остановить (тома сохраняются)
docker compose -f deploy/docker-compose.yml down -v      # остановить и УДАЛИТЬ данные
```

## 11. Безопасность (чек-лист перед эксплуатацией)

- ☐ Сменены пароли `POSTGRES_PASSWORD`, `MINIO_ROOT_PASSWORD`, а также `MINIO_ROOT_USER` (значение из примера — `minioadmin`).
- ☐ Задан свой `JWT_KEY` ( $\geq 32$  символов, случайный). Без этого приложение не запустится.
- ☐ Заданы свои `MINIO_ROOT_USER` / `MINIO_ROOT_PASSWORD`, `MINIO_BUCKET` и все параметры БД (`POSTGRES_DB` / `POSTGRES_USER` / `POSTGRES_PASSWORD`). Значения из файла-примера приложение тоже не примет — запуск останавливается с указанием переменной.
- ☐ Порты БД/MinIO/Ollama не опубликованы в интернет. Консоль MinIO поставляемый `docker-compose.yml` публикует только на петлю — убедитесь, что `MINIO_CONSOLE_BIND` не переопределён на `0.0.0.0` (см. §6).
- ☐ Том `dp_keys` защищён как секрет: доступ к нему (или к его копии) позволяет выписать действительную ссылку сброса пароля на любую учётную запись, минуя пароль и блокировку. В общие резервные копии его не кладут; если кладут — хранят вместе с прочими секретами. **Он же расшифровывает ключи интеграций** (см. §7): при потере тома ключи распознавания, веб-поиска и пароль SMTP придётся ввести заново — данные и документы при этом не страдают.



- ☐ **HTTPS обязателен**, если система доступна не только с локальной машины: перед `web` стоит обратный прокси с терминацией TLS, порт `80` наружу закрыт. Приложение TLS не терминирует и само на HTTPS не перенаправляет — без внешнего контура пароли и токены идут открытым текстом.
- ☐ Создан администратор; лишние тестовые учётные записи удалены.
- ☐ Настроено регулярное резервное копирование БД и `minio_data`.
- ☐ Известно, где смотреть вес встроенной резервной копии против предела восстановления (**Настройки** → **Резервное копирование**). Вес задаётся библиотекой документов качества и растёт вместе с ней; предел поднимается переменной `BACKUP_MAX_ARCHIVE_MB`.

---

## Приложение А. Установка Docker

---

§2 требует **Docker Engine 24+** и **Compose v2 или новее**, но на чистом сервере их ещё нет. Здесь — путь от голой ОС до состояния, в котором «Быстрый старт» (§3) проходит целиком.

Ставим **только из официального репозитория Docker**: так одной установкой получают и свежий движок, и `docker compose` как его плагин. Пакеты из репозитория дистрибутивов и `snar` тоже называются «докер», но требованиям §2 не отвечают — чем именно это кончается, в А.4.

Команды ниже прогнаны на чистых Ubuntu 24.04 и Rocky Linux 9 (август 2026): из репозитория приходят Engine 29.x и плагин Compose 5.x. Официальная страница установки со временем меняется (формат файла репозитория здесь уже менялся) — если команда не сработала, сверьтесь с <https://docs.docker.com/engine/install/>.

### А.1. Ubuntu 22.04+ / Debian 12+

Порядок один и тот же, отличается одна переменная в начале.

```

DISTRO=ubuntu          # для Debian: DISTRO=debian

# 1. Снять пакеты, которые займут имена команд и будут мешать.
#   Часть из них в системе не стоит — сообщения «не найден» здесь ожидаемы.
for p in docker.io docker-doc docker-compose docker-compose-v2 \
        docker-buildx podman-docker containerd runc; do
    sudo apt remove -y "$p" 2>/dev/null || true
done

# 2. Ключ и репозиторий Docker
sudo apt update
sudo apt install -y ca-certificates curl
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL "https://download.docker.com/linux/$DISTRO/gpg" -o /etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc

sudo tee /etc/apt/sources.list.d/docker.sources >/dev/null <<EOF
Types: deb
URIs: https://download.docker.com/linux/$DISTRO
Suites: $(. /etc/os-release && echo "${UBUNTU_CODENAME:-$VERSION_CODENAME}")
Components: stable
Architectures: $(dpkg --print-architecture)
Signed-By: /etc/apt/keyrings/docker.asc
EOF

# 3. Установка
sudo apt update
sudo apt install -y docker-ce docker-ce-cli containerd.io \
                    docker-buildx-plugin docker-compose-plugin

```

Подстановки внутри <<EOF выполняет ваша оболочка, а не `sudo` : в файл попадают уже готовые кодовое имя выпуска и архитектура. Стоит на них взглянуть — `cat /etc/apt/sources.list.d/docker.sources`. Пустое `Suites`: означает, что `/etc/os-release` не назвал кодовое имя (бывает на производных сборках); подставьте его вручную — `jammy`, `noble`, `bookworm`.

Дальше — A.3.

## A.2. RHEL-семейство: Rocky Linux 9 / AlmaLinux 9 / CentOS Stream 9

```

sudo dnf -y install dnf-plugins-core
sudo dnf config-manager --add-repo https://download.docker.com/linux/centos/docker-ce.repo
sudo dnf -y install docker-ce docker-ce-cli containerd.io \
                    docker-buildx-plugin docker-compose-plugin

```

На самой **RHEL** адрес другой — `https://download.docker.com/linux/rhel/docker-ce.repo`. Rocky и Alma собираются из исходников RHEL, но отдельного репозитория у Docker для них нет: им предназначен именно `centos`.

**firewalld**. В RHEL-семействе он включён по умолчанию — и опубликованные Docker порты он **не закрывает**: правила Docker стоят раньше. С `ufw` в Ubuntu ровно так же. Ограничивать доступ нужно там, где порт публикуется: привязкой к адресу в `docker-compose.yml` (как сделано для консоли MinIO, см. §6) или фильтром периметра. И проверять снаружи, а не выводом `firewall-cmd --list-ports`: список честно покажет, что порт не открыт, а `curl` с соседней машины в это же время откроет страницу.

**SELinux** трогать не нужно: поставка работает с именованными томами, метки для них расставляются сами. Понадобится, только если добавите свой `bind-mount`, — тогда ему нужен суффикс `:z` в описании тома.

### А.3. После установки — оба варианта

```
# На Debian/Ubuntu службу запустил и включил сам пакет — команда ничего не изменит.
sudo systemctl enable --now docker

# Чтобы не писать sudo перед каждой командой (о цене — сразу под блоком).
sudo usermod -aG docker "$USER"
newgrp docker                                # применить группу без перелога (в этой оболочке)
```

 **Группа `docker` — это `root` на хосте**, без оговорок: её участник запускает контейнер, смонтировав в него корень файловой системы, и дальше волен делать что угодно. Не «почти `root`» и не «`root` внутри контейнера» — доступ ко всему серверу. Включайте в неё только тех, кому и так дали бы `sudo` без пароля; на сервере с несколькими пользователями разумнее оставить `sudo docker ...`. (У Docker есть `rootless`-режим, снимающий это ограничение; наша поставка на нём не проверялась.)

**Если сервер выходит в интернет через прокси**, задать его нужно демону: переменные вашей оболочки Docker не наследует, и без этого `docker compose up` останавливается на скачивании образов, жалуясь на таймаут — то есть на что угодно, только не на прокси.

```
sudo mkdir -p /etc/systemd/system/docker.service.d
sudo tee /etc/systemd/system/docker.service.d/http-proxy.conf >/dev/null <<'EOF'
[Service]
Environment="HTTP_PROXY=http://proxy.example:3128"
Environment="HTTPS_PROXY=http://proxy.example:3128"
Environment="NO_PROXY=localhost,127.0.0.1"
EOF
sudo systemctl daemon-reload && sudo systemctl restart docker
```

### А.4. Чего не ставить

| Что                                                                                          | Чем это кончится                                                                                                                                                                                                                                                                       |
|----------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>apt install docker.io</code><br>— пакет из репозитория дистрибутива                    | Движка может хватить, а плагина Compose в нём нет: <code>docker compose version</code> отвечает <code>'compose' is not a docker command</code> . Половина команд этой инструкции просто не существует, причём выглядит это как опечатка в инструкции, а не как незаконченная установка |
| <code>docker-compose v1</code> (через <code>pip</code> или отдельным бинарником — с дефисом) | Другая программа: реализация на Python, снята с поддержки. Наш <code>docker-compose.yml</code> написан по спецификации Compose v2 (в нём намеренно нет ключа <code>version:</code> ), и v1 такой файл не понимает — падает на разборе, называя незнакомые поля                         |
| snap-пакет <code>docker</code>                                                               | Работает в изоляции snap: свои пути к данным и запрет на чтение файлов вне домашнего каталога. Отказы приходят как «нет прав» или «файл не найден» — по ним причину не угадать                                                                                                         |

Общее у всех трёх: они дают ошибки, не похожие на свою причину. Если `docker` в системе уже стоял и вы не знаете откуда — проверьте `docker compose version` (плагин обязан отвечать) и `dpkg -l | grep -i docker / rpm -qa | grep -i docker`.

## A.5. Проверка

```
docker --version          # 24.0 или новее (сегодня из репозитория ставится 29.x)
docker compose version    # плагин: через пробел, не через дефис
docker run --rm hello-world # запуск контейнера целиком, от скачивания до вывода
```

`hello-world` скачивается с Docker Hub, а образы системы — с `ghcr.io`. Успех этой команды означает «Docker работает», но не «до GHCR дотянемся»: это покажет уже `docker compose up`.

Дальше — **§3 «Быстрый старт»**.